# Codex i Claude Code

# Lokalna konfiguracja od podstaw

## Instrukcje. Skills. Subagenci. Hooks. MCP. Uprawnienia.

Praktyczny podręcznik na podstawie branchów agents/codex i agents/claude w repozytorium Agentic AI. Z przykładami pól, przepływem zgody i sposobem sprawdzania działania.

Stan dokumentacji: 7 listopada 2026. Konfiguracja jest wersjonowana; katalogi producentów pozostają źródłem informacji o nowych polach. Poradnik odróżnia konfigurację zapisaną od aktywnej oraz testy lokalne od testów interfejsu hosta.

# Spis treści


Codex i Claude Code 1

Spis treści 2

1. Najpierw pomysł: kto wykonuje pracę 3

2. Co zawierają nasze branche 4

3. AGENTS.md: stałe zasady pracy 5

4. Skill: procedura wielokrotnego użycia 6

5. Codex: pola i użycie skilla 7

6. Claude: pola i argumenty skilla 8

7. Subagent: odpowiedzialność i osobny kontekst 9

8. Codex: konfiguracja subagentów 10

9. Claude: konfiguracja subagentów 11

10. Hook: kod uruchamiany przy zdarzeniu 12

11. Hooki Codex: pola i ograniczenia 13

12. Hooki Claude: natywne pytania 14

13. Ustalona polityka operacji 15

14. Tool i MCP: własna funkcja dla hosta 16

15. Rejestracja MCP i zgoda w Codex 17

16. MCP w Claude i wspólne środowisko 18

17. Poziomy konfiguracji Codex 19

18. Poziomy konfiguracji Claude 20

19. Uprawnienia, sandbox i zaufanie 21

20. Jak dodawać własne elementy 22

21. Weryfikacja od pliku do interfejsu 23

22. Diagnostyka bez zgadywania 24

23. Plan krótkiego szkolenia 25

24. Źródła, wersje i dalsza rozbudowa 26

25. Rozszerzony katalog: hooki, narzędzia, pluginy 27

26. Przepisy na zakres repo, user i machine 28

27. Stan sprawdzenia tej konfiguracji 29

Aneks: pliki codex 30

Aneks: pliki claude 33

# 1. Najpierw pomysł: kto wykonuje pracę

Wyobraź sobie polecenie: „Przejrzyj lekcję, a potem zapisz pięć żartów”. Model rozumie zadanie i proponuje kroki. Aplikacja, w której model pracuje, dostarcza narzędzia i rozstrzyga ich wykonanie. Tę aplikację będziemy nazywać hostem. W naszym przykładzie hostem jest Codex albo Claude Code uruchomiony lokalnie, także przez rozszerzenie VS Code.

Model nie zapisuje pliku samym zdaniem „zapisałem”. Musi wywołać narzędzie. Zapis jest faktem dopiero wtedy, gdy narzędzie wykona operację, a odczyt potwierdzi wynik. Analogicznie plik konfiguracyjny nie działa dlatego, że agent go przeczytał: host musi go odnaleźć, zaakceptować i zastosować.

## Siedem elementów, siedem odpowiedzialności

| Element | Co robi w naszym repo |
|---|---|
| AGENTS.md | Ustala język, styl lekcji, importy i zasady pracy |
| Skill | Podaje powtarzalny sposób wyjaśniania lub oceniania lekcji |
| Subagent | Dostaje osobny kontekst i konkretną odpowiedzialność |
| Tool | Wykonuje operację o określonych argumentach |
| MCP | Udostępnia nasze narzędzia hostowi przez standardowy protokół |
| Hook | Reaguje kodem na zdarzenie, np. proponowane wywołanie |
| Permissions i sandbox | Kontrolują zgodę oraz dostęp do zasobów |

**Jak o tym opowiedzieć:** „Instrukcja mówi, jak pracować. Skill opisuje procedurę. Subagent realizuje wydzielone zadanie. Tool robi konkretną rzecz. Hook sprawdza zdarzenie, a host pilnuje zgody i uprawnień”.

## Mały przykład i wynik

`count_words(text="small useful example")` zwraca `3`. Nazwa funkcji identyfikuje narzędzie, `text` jest argumentem, a `3` wynikiem. Do tej operacji nie potrzeba skilla ani subagenta. Dodajemy je wtedy, gdy pomagają organizować zadanie, nie dlatego, że każdy przykład musi zawierać wszystkie mechanizmy.

# 2. Co zawierają nasze branche

Mamy dwa warianty kursowego repozytorium: **agents/codex** oraz **agents/claude**. Każdy ma własny README i instrukcję startową dla swojego hosta. Wspólne są reguły kursu, sens przykładów i implementacja lokalnego MCP. Katalogi konfiguracji hostów są rozdzielone.

| Codex | Claude Code |
|---|---|
| .codex/config.toml | .claude/settings.json oraz .mcp.json |
| .codex/hooks.json | hooks w .claude/settings.json |
| .codex/hooks/*.py | .claude/hooks/*.py |
| .codex/agents/*.toml | .claude/agents/*.md |
| .agents/skills/*/SKILL.md | .claude/skills/*/SKILL.md |
| docs/agents-codex.md | docs/agents-claude.md |
| docs/create-codex.md | docs/create-claude.md |

Oba mają `docs/coding-agent-guide.md`, `AGENTS.md` oraz `agent_demo/`. W każdym branchu są trzy skillsy i trzy role. Nie ma konieczności dokładania trzeciego aktywnego brancha tylko po to, by przechowywać wspólne instrukcje. Wspólne zmiany można przenosić małym, wydzielonym commitem wykonanym przez człowieka.

## Co jest wspólne, a co lokalne

W repo powinny być procedury, definicje ról, hooki, kod narzędzi i lockfile. Lokalnie zostają logowanie, sekrety, środowiska Python, zaufanie do projektu oraz osobiste ustawienia. Sklonowanie repo nie kopiuje decyzji zaufania innego użytkownika.

Pracujemy w osobnych checkoutach `agentic_ai-codex` i `agentic_ai-claude`. To ogranicza przypadkowe przełączanie konfiguracji pod uruchomioną rozmową. Branch jest nazwą historii Git, checkout jest rzeczywistym katalogiem z plikami. Niezacommitowane poprawki istnieją w checkoutcie, ale nie zostaną przekazane drugiej osobie samą nazwą brancha.

**Praktyka zespołowa:** po ręcznym commicie sprawdź czysty status i przekazuj nazwę brancha wraz z instrukcją uruchomienia. Nie scalaj całego wariantu Claude do Codex, jeśli chcesz utrzymać izolację katalogów hostów.

# 3. AGENTS.md: stałe zasady pracy

Nasz główny AGENTS.md jest krótki. Nakazuje najpierw wyjaśniać ideę, potem pokazywać niewielki przykład i wynik. Teksty repo oraz komentarze są po angielsku. Importy Python mają być na poziomie modułu. Istniejące zmiany użytkownika, historyczne materiały i lokalne środowiska trzeba zachować.

To dobre miejsce na regułę obowiązującą przy wielu zadaniach. Szczegółowy algorytm oceny lekcji należy do skilla, a kod kontroli wywołań do hooka. Gdy AGENTS.md staje się wielkim podręcznikiem każdej funkcji, każda rozmowa płaci kosztem dodatkowego kontekstu.

```
# Repository rules
- Explain the idea before the implementation.
- Keep imports at module scope.
- Preserve existing user changes.
- Do not create Git commits.
```

## Zakres i hierarchia

Codex wspiera instrukcje użytkownika oraz instrukcje w drzewie projektu. Bliższe danemu katalogowi zasady doprecyzowują pracę w tym obszarze; istnieje również wariant `AGENTS.override.md`. Nie oznacza to, że repo może uchylić politykę hosta albo administratora. W naszym repo notebooki i Acme mają własne wskazane pliki instrukcji.

Aktualny Claude Code obsługuje AGENTS.md w swoim mechanizmie pamięci projektu. Obecność konkurencyjnych plików CLAUDE.md lub CLAUDE.local.md może zmienić wybór źródła. Dlatego w przykładzie świadomie nie dodajemy drugiego, dublującego CLAUDE.md. Osobisty plik użytkownika nie jest tym samym co plik projektu.

## Czego instrukcja nie zapewnia

Zdanie „nie kasuj plików” wpływa na zachowanie modelu. Samo nie odbiera systemowi operacyjnemu możliwości usunięcia pliku. W naszym przykładzie instrukcję wspierają hooki; przy wymaganiach bezpieczeństwa potrzebna jest także kontrola uprawnień i dostępnych narzędzi.

**Ćwiczenie:** wybierz trzy reguły, które mają obowiązywać każdą zmianę w twoim repo. Jeśli reguła opisuje długą procedurę tylko jednego zadania, przenieś ją do skilla. Dokumentacja Codex i Claude.

# 4. Skill: procedura wielokrotnego użycia

Skill jest katalogiem z plikiem SKILL.md. Nagłówek YAML opisuje, co to za umiejętność i kiedy ją wybrać. Treść zawiera instrukcje. Dodatkowe pliki mogą przechowywać skrypty, przykłady, wzorce odpowiedzi i materiały referencyjne.

```
---
name: explain-code
description: Explain a named file using one concrete input and result.
---

Read the file and its relevant callers.
Show one input, the main steps, and the resulting output.
Separate observed behavior from assumptions. Do not edit the file.
```

`name` nie jest nazwą funkcji Python. `description` pomaga wybrać procedurę. Treść po separatorze nie jest automatycznie wykonywanym skryptem. Model ją czyta i realizuje przy pomocy dostępnych narzędzi, nadal podlegających uprawnieniom.

## Nasze trzy procedury

• **explain-code:** prowadzi jeden konkretny input przez rzeczywisty kod. Wynik powinien pokazać, co robi program i gdzie to widać.

• **review-lesson:** sprawdza kolejność pojęć, wymagania wstępne, ćwiczenia i czytelność przykładu. Wynik to uwagi z odwołaniem do materiału.

• **inspect-lesson:** zleca przegląd specjaliście. Uczy różnicy między procedurą a wykonawcą tej procedury.

## Trzy różne rodzaje „argumentów”

Pierwszy to pola konfiguracji skilla, np. `description`. Drugi to dane zadania przekazane podczas użycia, np. ścieżka notebooka i poziom odbiorcy. Trzeci to argumenty wywoływanego narzędzia, np. `file_path` i `jokes`. Własne pole YAML `audience: beginner` nie musi zostać rozpoznane przez hosta tylko dlatego, że wygląda sensownie.

Najlepszy skill określa wejście, kroki, format wyniku i warunek zatrzymania. Unikaj wielu kopii tych samych reguł, gdy można wskazać jeden plik referencyjny. Skrypt dołączony do skilla podlega takim samym kontrolom jak każdy inny uruchamiany program.

# 5. Codex: pola i użycie skilla

W tym branchu skill trafia do `.agents/skills/nazwa/SKILL.md`. Minimalny nagłówek zawiera `name` oraz `description`. Wywołanie w rozmowie ma postać `$explain-code model_config.py`. Ścieżkę i odbiorcę opisujemy jako wejście w treści procedury.

## Dodatkowy plik agents/openai.yaml

To plik wewnątrz katalogu konkretnego skilla, np. `.agents/skills/explain-code/agents/openai.yaml`. Nie myl go z `.codex/agents`, gdzie są osobne role subagentów.

| Pole | Zastosowanie |
|---|---|
| interface.display_name | Nazwa wyświetlana użytkownikowi |
| interface.short_description | Krótki opis w interfejsie |
| interface.default_prompt | Sugerowany tekst uruchomienia |
| interface.icon_small / icon_large | Względne ścieżki grafik |
| interface.brand_color | Kolor identyfikacji |
| policy.allow_implicit_invocation | false: procedurę uruchamia się jawnie |
| dependencies.tools[] | Opis wymaganych połączeń, np. MCP |

```
interface:
  display_name: "Explain code"
  short_description: "Follow one input through real code"
policy:
  allow_implicit_invocation: false
```

Obiekt zależności narzędzia może opisywać `type`, `value`, `description`, `transport` oraz `url`. Nie definiuje on schematu argumentów funkcji MCP. Schemat narzędzia jest dostarczany przez serwer.

## Jak rozpoznać błąd

Jeżeli skill nie jest wykrywany, sprawdź nazwę SKILL.md, poprawność YAML, właściwy katalog i świeżość sesji. Nie dodawaj odruchowo pól z przykładów Claude. `context: fork` nie jest tutaj przełącznikiem uruchomienia subagenta; nasz inspect-lesson opisuje delegację instrukcjami.

Skills użytkownika i skills projektu mają inne zasięgi. Ten sam opis skopiowany do obu miejsc komplikuje diagnostykę. Na początek utrzymuj jedną aktywną definicję o danej nazwie. Aktualny format Codex.

# 6. Claude: pola i argumenty skilla

Wariant Claude korzysta z `.claude/skills/nazwa/SKILL.md`. Użytkownik może wywołać `/explain-code model_config.py`. Claude ma więcej pól sterujących samym uruchomieniem procedury niż nasz wariant Codex.

| Pole lub grupa | Znaczenie |
|---|---|
| name, description, when_to_use | Tożsamość i wskazówki wyboru |
| argument-hint, arguments | Podpowiedź składni i nazwy pozycji |
| disable-model-invocation | true wymaga jawnego wywołania |
| user-invocable | false wyłącza wywołanie użytkownika |
| allowed-tools, disallowed-tools | Tymczasowe przyznania / usunięcia narzędzi |
| model, effort | Wybór modelu i obsługiwanego wysiłku |
| context, agent, background | Delegacja: fork, rola, oczekiwanie w tle |
| hooks | Obsługa zdarzeń związana z uruchomieniem skilla |
| paths, shell | Zakres ścieżek i powłoka wstrzykiwanych poleceń |
| metadata, license, compatibility | Dane opisowe, licencja, wymagania |

```
name: inspect-lesson
description: Review a lesson through the lesson specialist.
argument-hint: "<notebook-path> <audience>"
disable-model-invocation: true
context: fork
agent: lesson-reviewer
background: false
```

W treści można odwołać się do `$ARGUMENTS`, do pozycji `$0` lub `$ARGUMENTS[0]`, a przy nazwanych `arguments` do odpowiedniej nazwy. `${CLAUDE_SKILL_DIR}` pomaga znaleźć pliki dołączone do skilla. Podpowiedź argument-hint nie waliduje danych: brak wymaganej ścieżki trzeba obsłużyć w procedurze.

**Ważna różnica:** allowed-tools w skillu nie oznacza tego samego co tools w subagencie. Nie traktuj go automatycznie jako wyłącznej listy dostępnych operacji. Pola skills używają także innej pisowni, np. `disallowed-tools`, podczas gdy definicja agenta używa `disallowedTools`.

Zaawansowane wstrzykiwanie wyników poleceń do treści skilla jest wykonaniem kodu. Zaczynaj od statycznej procedury; dodaj je tylko wtedy, gdy świeży wynik ma praktyczną wartość. Pełny bieżący kontrakt.

# 7. Subagent: odpowiedzialność i osobny kontekst

Subagent jest dodatkowym przebiegiem modelu, a nie skryptem Python. Dostaje zadanie, instrukcje roli i dostępne narzędzia. Główny agent odbiera wynik, ale nie powinien udawać, że delegacja się odbyła, gdy brak rzeczywistego uruchomienia specjalisty.

Nasze role są proste: dokumentacyjny reviewer sprawdza zgodność tekstu z plikami; lesson reviewer ocenia dydaktykę; joke writer proponuje i zapisuje pięć żartów po zgodzie. Dwie pierwsze role mają raportować ustalenia bez edycji. Trzecia odpowiada za konkretny artefakt.

## Dobry kontrakt delegacji

Zlecenie powinno wskazać plik, cel, dozwoloną odpowiedzialność i oczekiwany wynik. „Sprawdź wszystko” daje znacznie mniej przewidywalną pracę niż „Przeczytaj jeden notebook, znajdź trzy największe luki dla początkującego i podaj numery komórek”.

```
Use lesson-reviewer to inspect
notebooks/langchain/01_create_agent.ipynb
for a reader who knows basic Python.
Return evidence-backed teaching issues. Do not edit or execute cells.
```

Przy równoległej pracy nad kodem każdy wykonawca powinien mieć własny zakres plików. Wspólny katalog roboczy oznacza, że zmiany innych są od razu widoczne. Oddzielny kontekst rozmowy nie daje automatycznie oddzielnego filesystemu.

## Zapis żartów bez pośrednika

Joke writer sam wywołuje `write_jokes`, czeka na decyzję hosta i czyta wynik. Rodzic może potem przekazać użytkownikowi rezultat. Nie jest potrzebny plik „zgoda.txt”, numer żądania zapisany przez rodzica ani ponowne wykonywanie operacji przez agenta głównego.

Odmowa zamyka tę próbę. Zmiana narzędzia po odmowie byłaby obejściem decyzji, a nie rozwiązaniem błędu technicznego. Odróżnij ją od błędu zwykłego wejścia, np. przekazania czterech zamiast pięciu żartów.

# 8. Codex: konfiguracja subagentów

Role zapisujemy jako pliki TOML w `.codex/agents`. Nazwy naszego wariantu używają podkreśleń: docs_reviewer, lesson_reviewer, joke_writer. Minimalna definicja jest krótka:

```
name = "docs_reviewer"
description = "Check documentation against actual repository files."
developer_instructions = """
Read the named document and the files it describes.
Return confirmed discrepancies with paths. Do not edit.
"""
```

| Pole | Co kontroluje |
|---|---|
| name, description | Tożsamość roli i dobór do zadania |
| developer_instructions | Instrukcje specjalisty |
| model | Opcjonalny model; brak zachowuje dziedziczenie |
| model_reasoning_effort | Obsługiwany przez model poziom rozumowania |
| model_reasoning_summary, model_verbosity | Preferencje odpowiedzi |
| features.* | Wyłączenie wybranych możliwości roli |
| skills.config / bundled.enabled / include_instructions | Sterowanie dostępem i instrukcjami skills |

Wspierane przełączniki roli obejmują m.in. shell_tool, apps, plugins, memories i request_permissions_tool. Nie są ogólnym odpowiednikiem Claude tools. Pola dotyczące sandboxa, zatwierdzania i serwerów MCP nie służą tu do zastąpienia polityki odziedziczonej po rodzicu.

```
[agents]
enabled = true
max_concurrent_threads_per_session = 2
```

Ten fragment jest w głównym `.codex/config.toml`, nie w pliku pojedynczej roli. Oprócz niego sekcja agents może sterować domyślnym modelem subagenta, jego wysiłkiem oraz komunikatem przerwania. W naszym repo pozostawiamy model odziedziczony, żeby nie wiązać lekcji z jednym kontem lub modelem.

**Granica gwarancji:** read-only zapisane w developer_instructions jest instrukcją zachowania. W tym przykładzie nie twierdzimy, że reviewer dostał osobny, technicznie niezmienialny filesystem. Dokumentacja subagentów Codex.

# 9. Claude: konfiguracja subagentów

Claude używa Markdown z nagłówkiem YAML w `.claude/agents`. Nasze nazwy to docs-reviewer, lesson-reviewer i joke-writer. Treść dokumentu jest instrukcją roli, a nagłówek ustala jej parametry.

```
name: docs-reviewer
description: Check documentation against actual repository files.
tools: Read, Grep, Glob
model: inherit
```

| Pole | Funkcja |
|---|---|
| name, description | Wymagana tożsamość i opis |
| tools, disallowedTools | Dostępna pula narzędzi oraz wyłączenia |
| model, effort | Model / dziedziczenie i wysiłek |
| permissionMode | Tryb uprawnień, np. default lub plan |
| maxTurns, autoCompactWindow | Limit kroków i próg kompakcji |
| skills | Wstępnie dołączane procedury |
| mcpServers | Dostępne serwery: nazwy lub definicje |
| hooks | Hooki związane z rolą |
| memory | Pamięć user, project lub local |
| background, isolation | Praca w tle; izolacja worktree |
| omitClaudeMd | Pominięcie zwykłego kontekstu CLAUDE.md |
| color, initialPrompt | Prezentacja i start roli jako sesji |
| experimental.cacheTtl | Eksperymentalny czas cache |

Tutaj `tools: Read, Grep, Glob` faktycznie ogranicza pulę narzędzi specjalisty. Nasz joke-writer ma Read oraz konkretne MCP write_jokes. Nie potrzebuje dowolnego Bash ani Write, żeby stworzyć ten jeden artefakt.

Nie wybieramy trybów pomijających zgody. `permissionMode: default` zachowuje normalne interakcje hosta, a projektowy hook wymusza pytanie dla poprawnej propozycji żartów. Rodzic i subagent korzystają z tego samego kontrolowanego narzędzia.

Worktree oddziela checkout kodu, lecz nie tworzy automatycznie nowego konta systemowego ani hermetycznej piaskownicy. Pamięć agenta również wymaga świadomego wyboru miejsca: dane lokalne nie powinny przypadkiem trafić do wspólnej historii. Bieżące pola i ograniczenia.

# 10. Hook: kod uruchamiany przy zdarzeniu

Hook działa wtedy, gdy host emituje określone zdarzenie. W naszych przykładach są dwa momenty: przed narzędziem i po edycji. Handler Python dostaje JSON na stdin i zwraca JSON na stdout. Nie potrzebuje modelu językowego.

```
Propozycja narzędzia + argumenty
  -> PreToolUse: sprawdzenie reguł
  -> host: rozstrzygnięcie uprawnień i ewentualna zgoda
  -> wykonanie narzędzia
  -> PostToolUse: informacja po wykonaniu
```

Nie używamy hooka po wykonaniu jako zabezpieczenia przed zapisem, bo wtedy operacja już zaszła. PostToolUse jest dobry do przypomnienia o walidacji notebooka. Nie stanowi dowodu, że testy uruchomiono.

## Wejście i wynik

Przy zdarzeniu narzędziowym interesują nas `tool_name`, `tool_input` oraz katalog `cwd`. Narzędzie powłoki może przesłać pole command albo cmd. Patch może przyjść jako tekst lub obiekt. Handler musi obsługiwać format rzeczywiście używany przez hosta, nie tylko jeden ulubiony przykład testowy.

```
{"hookSpecificOutput": {
  "hookEventName": "PreToolUse",
  "permissionDecision": "deny",
  "permissionDecisionReason": "Commits are forbidden for agents."
}}
```

Pusty obiekt `{}` oznacza kontynuowanie normalnego rozstrzygania, a nie uniwersalną zgodę. Return allow może zmienić ścieżkę zatwierdzania, dlatego nie używamy go bez potrzeby. Wiadomości diagnostyczne kieruj na stderr; dodatkowy tekst na stdout może zepsuć protokół.

## Rejestracja a wykonanie

Plik Python bez wpisu w konfiguracji nie jest aktywnym hookiem. Wpis bez zaufania może być widoczny, lecz nieaktywny. Działający test handlera nie dowodzi, że host go uruchomił. Pełna weryfikacja wymaga także próby w świeżej sesji i obserwacji skutku.

# 11. Hooki Codex: pola i ograniczenia

Codex czyta projektowe definicje z `.codex/hooks.json`. Mapa hooks zawiera zdarzenia, każde zdarzenie grupy matcher/hooks, a grupa listę handlerów. Matcher naszych zdarzeń jest wyrażeniem regularnym dopasowanym do nazwy narzędzia.

| Pole | Znaczenie w naszym przykładzie |
|---|---|
| hooks.PreToolUse / PostToolUse | Moment uruchomienia |
| matcher | Regex nazwy narzędzia |
| hooks[].type | command: uruchomienie programu |
| command | Polecenie startowe |
| commandWindows | Wariant polecenia na Windows |
| timeout | Limit czasu w sekundach |
| statusMessage | Etykieta podczas działania |
| async | Praca asynchroniczna, niewłaściwa do blokowania operacji |
| additionalContextLimit | Limit przekazywanego kontekstu |

Nasze hooki decyzyjne są synchroniczne i mają 10 sekund. Polecenie przez Git znajduje katalog repo, więc działa także po uruchomieniu z notebooks/. Python startuje z `-I`, aby ograniczyć wpływ przypadkowych ustawień środowiska importów.

## Najważniejsza różnica dotycząca zgody

W sprawdzonej wersji Codex nie obsługuje `ask` jako decyzji PreToolUse. Wysłanie takiej wartości nie jest poprawnym sposobem wymuszenia pytania. Nie oznacza to, że cały Codex umie tylko odmawiać: natywną zgodę dla naszego narzędzia uzyskujemy ustawieniem MCP `approval_mode = "prompt"`.

Hook approve_jokes sprawdza wejście. Jeśli jest poprawne, zwraca `{}` i pozwala hostowi wyświetlić wymagany monit MCP. Jeśli jest błędne, odmawia. Dzięki temu walidacja nie zastępuje świadomie wymaganej zgody.

## Inne zdarzenia

Kontrakt hosta obejmuje także PermissionRequest, sesje, prompty i kompakcję. Każde zdarzenie ma własne wejście i dozwolone wyjście. Nie przenoś JSON-a z PreToolUse do innego zdarzenia bez sprawdzenia schematu. Hook PermissionRequest reaguje na istniejącą potrzebę zgody; nie należy utożsamiać go z dowolnym okienkiem pytania. Kontrakt i katalog zdarzeń.

# 12. Hooki Claude: natywne pytania

Claude rejestruje hooki w `.claude/settings.json`. W naszym wariancie command to `python`, a args jest tablicą bezpośrednich argumentów. To ogranicza problemy cytowania ścieżek w powłoce.

```
{"type": "command", "command": "python",
 "args": ["-I", "${CLAUDE_PROJECT_DIR}/.claude/hooks/change_policy.py",
          "--deletion-action", "ask"],
 "timeout": 10, "onFailure": "block"}
```

| Pole | Co ustawia |
|---|---|
| matcher, type | Selekcja zdarzenia i rodzaj handlera |
| command, args | Program i argv; args wyłącza interpretację jako polecenie shell |
| shell | Powłoka przy formie polecenia bez args |
| timeout, statusMessage | Limit i opis postępu |
| onFailure | block albo continue przy awarii |
| if | Dodatkowe filtrowanie zgodne z regułami uprawnień |
| async | Działanie w tle; nie dla naszych decyzji |
| once | Jednorazowość tam, gdzie obsługuje ją kontekst skilla |

## Wynik przed narzędziem

Claude rozumie `allow`, `deny`, `ask`, a w odpowiednich przepływach także `defer`. Nasza zgoda korzysta z ask. Powód zawiera pełną ścieżkę i treść proponowanego zapisu albo manifest usunięcia. Host wyświetla pytanie, a dopiero pozytywna decyzja pozwala wykonać oryginalną operację.

Konflikt kilku hooków nie oznacza, że ostatnia odpowiedź wygrywa. Odmowa ma wyższy priorytet niż prośba o zgodę. Dlatego propozycja destrukcyjnego Git nie może zostać dopuszczona tylko dlatego, że inny handler rozpoznał usuwanie i zwrócił ask.

## Interaktywność ma znaczenie

Tryb drukujący wynik bez interfejsu pytania może odmówić wywołania wymagającego zgody. To nie jest równoważne kliknięciu „Nie” przez człowieka. Nie usuwaj wymogu zgody tylko po to, by test nieinteraktywny zakończył się sukcesem.

Claude ma również inne typy hooków, m.in. HTTP oraz oparte o model lub narzędzie. Ich pola i dostępność zależą od typu i zdarzenia; nie są zamiennikami command z identycznym schematem. W repo celowo korzystamy z deterministycznego Pythona. Pełna referencja.

# 13. Ustalona polityka operacji

| Operacja | Codex | Claude Code |
|---|---|---|
| Zwykły commit | Odmowa | Odmowa |
| Destrukcyjny Git | Odmowa | Odmowa |
| Zapis pięciu żartów | Natywna zgoda MCP | Natywne ask z hooka |
| Istniejący jokes.txt | Bez nadpisania | Bez nadpisania |
| Usunięcie pliku | Odmowa + pełne ścieżki | Pytanie + pełne ścieżki |
| Niejasne cele usunięcia | Odmowa | Odmowa |

## Cztery odpowiedzialności kodu

`git_safety.py` wykrywa rozpoznane formy reset --hard, wymuszonego clean i push. `change_policy.py` obsługuje zwykłe commity, usuwanie oraz rozpoznane próby bezpośredniego zapisu jokes.txt. `approve_jokes.py` waliduje propozycję MCP. `notebook_context.py` przypomina o sprawdzeniu notebooka po edycji.

Handler zmiany nie wykonuje polecenia, które analizuje. Może więc dostać ciąg znaków opisujący niebezpieczny Git w teście jednostkowym bez wykonywania tego Git. Tak sprawdzamy przypadki odmowy.

## Manifest usuwania

Jawny `Remove-Item -LiteralPath 'one file.txt'` daje konkretną ścieżkę względem workdir narzędzia. Przy rekursji hook wylicza zawartość bez podążania za linkami katalogowymi. Lista jest ograniczona do 200 elementów; większe lub niejasne operacje trzeba najpierw rozpoznać tylko do odczytu i podzielić.

Zmienne, wildcardy i kasowanie ukryte wewnątrz kodu są odrzucane, gdy nie da się wyznaczyć rzetelnej listy. Ścieżka zdalnego zasobu nie powinna zostać udawana jako lokalny plik. Manifest jest migawką: między pytaniem a wykonaniem filesystem może się zmienić.

**Granica przykładu:** analizator tekstu nie rozumie wszystkich języków, aliasów, skryptów i dowolnych connectorów. To kontrola rozpoznanych wywołań, nie dowód niemożliwości obejścia przez dowolny kod. Przy twardym wymaganiu zakazu trzeba ograniczyć dostępne operacje i uprawnienia systemowe.

# 14. Tool i MCP: własna funkcja dla hosta

MCP jest protokołem połączenia hosta z serwerem narzędzi. W naszym repo serwer działa lokalnie jako proces Python przez stdio. Host wysyła wiadomości protokołu, serwer udostępnia definicje funkcji i odsyła wyniki.

```
from mcp.server import MCPServer

server = MCPServer("lesson_tools")

@server.tool()
def count_words(text: str) -> int:
    """Count whitespace-separated tokens in text."""
    return len(text.split())

if __name__ == "__main__":
    server.run()
```

Dekorator rejestruje funkcję. Nazwa jest identyfikatorem narzędzia, docstring opisuje jego sens, a adnotacje typów służą do utworzenia schematu. `str` mówi hostowi, że text ma być ciągiem znaków. Brak wymaganego argumentu ma skutkować błędem, a nie zgadywaniem wartości.

## Dwa narzędzia w jednej usłudze

`count_words` jest operacją odczytową. Dzieli tekst po białych znakach, więc interpunkcja pozostaje przy słowie. Nie jest językowym tokenizatorem modelu.

`write_jokes(file_path: str, jokes: list[str]) -> str` tworzy jeden dozwolony plik. Ścieżka musi być absolutna i odpowiadać root jokes.txt. Tablica ma mieć pięć niepustych jednowierszowych tekstów. Serwer sam nadaje numerację 1-5.

Zapis używa `open("x")`: jeżeli plik powstał już po wyświetleniu pytania, operacja nadal odmówi nadpisania. To kontrola w samym narzędziu, niezależna od wcześniejszej walidacji hooka. Sam serwer nie wyświetla pytania hosta i nie zna kliknięcia użytkownika.

MCP może udostępniać także zasoby i prompty. W tym repo ich nie dodajemy, bo dwa narzędzia wystarczają do zademonstrowania integracji. Nie ma zewnętrznego API ani sekretu potrzebnego do count_words. Oficjalny SDK Python.

# 15. Rejestracja MCP i zgoda w Codex

Serwer trzeba zarejestrować, aby host wiedział, jaki proces uruchomić. Nasz PowerShell odnajduje Git root i wykonuje uv z zablokowanymi zależnościami. Dzięki temu zmiana katalogu startowego na notebooks/ nie kieruje procesu do przypadkowego pliku.

```
[mcp_servers.lesson_tools]
command = "powershell.exe"
# args contains the repository-root launcher
enabled_tools = ["count_words", "write_jokes"]
startup_timeout_sec = 30
tool_timeout_sec = 10

[mcp_servers.lesson_tools.tools.write_jokes]
approval_mode = "prompt"
```

| Grupa pól | Do czego służy |
|---|---|
| command, args, cwd | Proces stdio i jego katalog |
| env, env_vars | Środowisko procesu i przekazywane zmienne |
| enabled, enabled_tools, disabled_tools | Włączenie serwera i wybór narzędzi |
| startup_timeout_sec, tool_timeout_sec | Limity startu i wywołania |
| default_tools_approval_mode | Domyślna polityka narzędzi serwera |
| tools.NAZWA.approval_mode | Polityka konkretnego narzędzia |
| url, bearer_token_env_var | Połączenie HTTP i źródło tokenu |

Dostępne wartości trybu obejmują auto, prompt, writes oraz approve. Tutaj wybór jest konkretny: write_jokes ma prompt. Tryb writes opiera się na klasyfikacji narzędzi jako odczytowych lub zapisujących; jawne ustawienie pojedynczego narzędzia lepiej pokazuje nasz wymagany przykład.

## Przepływ zapisu

Subagent przygotowuje argumenty. Hook odrzuca niepoprawny cel lub treść; poprawne wejście przechodzi dalej. Codex prosi o zgodę na write_jokes. Po akceptacji MCP wykonuje funkcję. Po odmowie nie ma pliku i specjalista kończy zadanie.

W natywnym teście zobaczyliśmy pytanie „Allow the lesson_tools MCP server to run tool write_jokes?” po uruchomieniu subagenta. To odrębny dowód od testu parsowania TOML. Aktualne pola MCP.

# 16. MCP w Claude i wspólne środowisko

Claude zapisuje projektowe serwery w `.mcp.json`, pod kluczem mcpServers. Nasz wpis ma typ stdio, program PowerShell i argumenty uruchamiające ten sam Pythonowy serwer. Użytkownik musi zaakceptować połączenie projektu we własnym hoście.

```
{"mcpServers": {"lesson_tools": {
  "type": "stdio",
  "command": "powershell.exe",
  "args": ["-NoProfile", "-NonInteractive", "-Command", "..."]
}}}
```

Wielokropek pokazuje miejsce launchera, nie jest poleceniem do skopiowania. Gotowy, kompletny argv znajduje się w pliku brancha. Nie dopisuj `${CLAUDE_PROJECT_DIR}` wszędzie przez analogię: interpolacja argumentów hooka i konfiguracja MCP mają odrębne kontrakty.

## Zakresy serwera

Project udostępnia konfigurację zespołowi przez repo. User udostępnia połączenie użytkownikowi w wielu projektach. Local dotyczy prywatnej konfiguracji jednego projektu. Pozornie identyczny serwer może być widoczny z różnych źródeł; w diagnostyce sprawdź źródło, nie tylko nazwę.

Dla stdio istotne są command, args i env. HTTP używa type, url oraz ewentualnych headers lub mechanizmu uwierzytelnienia. Nie zapisuj tokenów w śledzonym JSON. Host może oferować konfigurację OAuth; to osobny proces od zatwierdzania pojedynczego zapisu pliku.

## Izolacja zależności od kursu

`agent_demo/pyproject.toml` i uv.lock tworzą mały, niezależny projekt. Środowisko `agent_demo/.venv` nie zastępuje głównego środowiska notebooków. `uv sync --frozen --project agent_demo` odtwarza zapisany lock, a test check_server uruchamia dokładnie zarejestrowany proces.

Połączenie stdio używa stdout do wiadomości MCP. Logowanie zwykłym print na tym kanale może uszkodzić komunikację. Błędy diagnostyczne kieruj na stderr. Instalacja zależności może wymagać internetu, choć samo liczenie słów i zapis żartów są lokalne. MCP w Claude.

# 17. Poziomy konfiguracji Codex

Najpierw ustal, kogo ma dotyczyć zmiana. Reguła zespołowa należy do repo. Preferencja osobista należy do konfiguracji użytkownika. Ograniczenie organizacji należy do mechanizmów zarządzanych. Jednorazowa decyzja w rozmowie nie powinna bez potrzeby stawać się globalną regułą.

| Poziom | Przykład i skutek |
|---|---|
| Projekt | .codex/config.toml; przenoszony razem z repo |
| Użytkownik | config.toml w katalogu Codex użytkownika |
| Instrukcje użytkownika | Osobiste AGENTS.md w katalogu konfiguracji |
| Sesja / CLI | Parametry bieżącego uruchomienia |
| Managed | Wymagania administratora ograniczające dopuszczalne wybory |
| System operacyjny | ACL, procesy, środowisko, dostęp do sieci |

Na tym komputerze katalog użytkownika Codex jest pod `C:/Users/Komputer/.codex`. Ta ścieżka jest przykładem lokalnym, a nie ścieżką do skopiowania na komputer kolegi. Uwzględniaj CODEX_HOME i wybrane przez organizację rozmieszczenie konfiguracji.

## Kolejność rozstrzygania

Nie mieszaj dwóch hierarchii: składania preferencji i narzuconych wymagań. Ustawienie sesji lub projektu może zmienić zwykłą preferencję, ale nie ma automatycznie prawa uchylić ograniczenia zarządzanego. Zaufanie do projektu warunkuje ładowanie jego aktywnej konfiguracji.

Model, approval_policy, sandbox_mode, profile i features należą do konfiguracji hosta. W naszym branchu pozostawiamy je odziedziczone, poza minimalnym włączeniem subagentów i MCP. Nie ma tu „magicznego pełnego dostępu”, który musisz globalnie włączać, aby zrozumieć lekcję.

## Jak ustalić efektywny wynik

Sprawdź wersję uruchomionego runtime, katalog startowy, źródła konfiguracji i stan zaufania. Osobny codex z PATH może być starszy od binarki rozszerzenia VS Code. Zmiana tekstu na dysku nie dowodzi zmiany już otwartej sesji. Po zmianach uruchom świeżą rozmowę i sprawdź konkretną funkcję.

Aktualny katalog wszystkich ustawień hosta jest duży i obejmuje funkcje niezwiązane z naszym repo. Korzystaj z referencji config jako indeksu dokładnych typów; nie dopisuj podobnie brzmiących pól na podstawie intuicji.

# 18. Poziomy konfiguracji Claude

Claude rozdziela ustawienia współdzielone i lokalne osobnym plikiem. To przydatne, kiedy chcesz mieć wspólne hooki w repo, ale inny model albo osobiste preferencje tylko u siebie.

| Zakres | Plik / miejsce |
|---|---|
| User | ~/.claude/settings.json |
| Shared project | .claude/settings.json |
| Project local | .claude/settings.local.json |
| Managed | Polityka dostarczana przez organizację |
| Jedna sesja | Flagi CLI lub właściwe ustawienia sesji |
| MCP project | .mcp.json, odrębny format od settings |

W zwykłej hierarchii najwyżej są ustawienia managed, potem uruchomienie przez CLI, następnie lokalny projekt, współdzielony projekt i użytkownik. Niektóre pola mają dodatkowe zasady, a listy mogą się łączyć zamiast całkowicie zastępować. Dlatego dodanie allow w jednym pliku nie oznacza usunięcia deny z innego.

## Uprawnienia jako osobna część settings

Sekcja permissions może zawierać allow, ask, deny oraz defaultMode. Nasz przykład ustawia defaultMode na default. Reguły narzędziowe są dopasowaniami do narzędzi i argumentów, a nie dowolnym kodem Python. Hook obsługuje przypadki wymagające dodatkowej walidacji, takie jak wyliczenie pełnych ścieżek.

Ustawienia typu „tylko zarządzane hooki” lub ograniczenia serwerów mogą sprawić, że poprawny projektowy plik nie będzie aktywny. Nie naprawia się tego przez dodawanie coraz większej liczby podobnych wpisów. Trzeba sprawdzić faktyczne źródło ograniczenia.

## Co przekazać zespołowi

Śledź settings.json, definicje ról, skills i MCP projektu. settings.local.json oraz środowiska utrzymuj poza Git. Zaufanie i zgodę na projektowy MCP każdy użytkownik rozstrzyga u siebie. Sesja chmurowa działa na innej maszynie i nie dziedziczy automatycznie plików z twojego katalogu domowego.

Zakresy i pierwszeństwo oraz indeks wszystkich kluczy służą do rozwiązywania konfliktów. Przy aktualizacji hosta sprawdź także wyjątki dotyczące konkretnego pola.

# 19. Uprawnienia, sandbox i zaufanie

Te trzy słowa opisują różne pytania. Zaufanie odpowiada: czy host może aktywować kod i konfigurację tego projektu? Uprawnienia: czy proponowana operacja jest dopuszczona lub wymaga decyzji człowieka? Sandbox: jakie zasoby proces może rzeczywiście odczytać lub zmienić?

## Przykład z naszego środowiska

Główny katalog repo jest zapisywalny dla bieżącej sesji, ale sąsiednie checkouty mogą znajdować się poza jej zapisywalnym zakresem. Zapis do nich wymaga wtedy eskalacji samego środowiska pracy. To nie jest dowód, że projektowy hook pyta o każdą edycję.

Podobnie błąd dostępu do cache uv nie dowodzi błędu narzędzia MCP. Najpierw ustal warstwę: start procesu, zależności, transport, schemat, hook, zgoda czy logika funkcji. Test uruchomiony z prawidłowymi uprawnieniami może przejść bez zmiany kodu aplikacji.

## Co oznacza „bez dodatkowego pytania”

Nie oznacza automatycznie pełnego dostępu do komputera. Operacja może nadal trafić na zwykłe ograniczenie sandboxa lub ustawienie użytkownika. W tym przewodniku zapis żartów ma **jawnie wymaganą** zgodę, a nie tylko możliwość pojawienia się pytania przez przypadek.

Nie używamy globalnego pomijania uprawnień, aby pokazać pozytywny test. Nie zapisujemy sztucznego tokenu zgody do repo. W normalnej pracy pytanie pojawia się w hoście obsługującym wywołanie subagenta.

## Granica kontroli przez edytowalny projekt

Osoba mająca prawo zmienić hook i ponownie mu zaufać może zmienić jego zachowanie. Projektowy przykład nie jest nieusuwalną polityką organizacyjną. Jeśli potrzebujesz takiej gwarancji, zastosuj zarządzaną konfigurację, ograniczenia narzędzi oraz kontrolę systemową. To decyzja administracyjna, niezależna od jakości tekstu w AGENTS.md.

# 20. Jak dodawać własne elementy

Zacznij od zdania opisującego wejście i obserwowalny wynik. Na przykład: „Dla ścieżki notebooka zwróć listę komórek z importami poza pierwszą komórką”. Dopiero potem dobierz mechanizm.

## Procedura tworzenia

• Jeśli to powtarzalny sposób myślenia i raportowania, utwórz skill z krótkim nagłówkiem, krokami i formatem wyniku.

• Jeśli zadanie zyskuje na osobnym kontekście, utwórz rolę i nadaj jej konkretną odpowiedzialność.

• Jeśli wynik ma być deterministyczny, napisz funkcję i dopiero potem udostępnij ją jako tool.

• Jeśli reakcja ma nastąpić zawsze przy zdarzeniu hosta, dodaj hook z małym, sprawdzalnym kontraktem.

• Jeśli chodzi o dostęp, ustaw uprawnienia; nie próbuj zastępować ich coraz bardziej stanowczym promptem.

## Przykład rozszerzenia MCP

```
@server.tool()
def count_lines(text: str) -> int:
    """Count logical lines in supplied text without reading files."""
    return len(text.splitlines())
```

Funkcja przyjmuje tekst, a nie ścieżkę. To świadomie ogranicza zakres: nie potrzebuje dostępu do plików. Dodaj ją do enabled_tools, jeśli host używa listy dopuszczonych narzędzi. Zrestartuj serwer, sprawdź discovery i wywołaj pusty tekst oraz tekst wielowierszowy.

## Gdy dodajesz zapis

Określ dozwolone cele, zachowanie przy istniejącym pliku, format danych i potrzebę zgody. Waliduj w narzędziu także po zatwierdzeniu. Sprawdź odmowę oraz ponowną próbę. Nie traktuj sukcesu jednej ścieżki jako dowodu poprawności całej polityki.

W repo zachowaj tylko elementy, które pokazują konkretną różnicę lub rozwiązują realny problem. Dwa prawie identyczne skillsy z niejasnymi nazwami są trudniejsze do nauczenia niż jedna dobra procedura z dwoma przykładami wejścia.

## Kiedy potrzebujesz pluginu

Plugin jest pakietem dystrybucyjnym, który może łączyć skills, hooki i MCP. Nie trzeba tworzyć pluginu, żeby działał projektowy skill. Pakowanie ma sens przy dystrybucji wielu repozytoriom; dla naszego przykładu zwykłe pliki branchów są czytelniejsze.

# 21. Weryfikacja od pliku do interfejsu

Weryfikacja ma kilka poziomów. Każdy odpowiada na inne pytanie i żaden nie powinien być przedstawiany jako dowód poziomu, którego nie sprawdził.

| Poziom | Dowód |
|---|---|
| Składnia | JSON/TOML/YAML daje się sparsować |
| Handler | Konkretny event daje poprawną decyzję bez wykonania operacji |
| Rejestracja | Host widzi właściwy skill, rolę, hook lub MCP |
| Protokół | Serwer startuje, publikuje schemat i odpowiada |
| Natywna interakcja | Rzeczywisty host pokazuje pytanie lub odmawia |
| Efekt | Plik powstał po zgodzie albo pozostał po odmowie |

## Testy repo

Codex: test_coding_agent_patterns, test_codex_patterns, test_change_patterns i test_rag_lessons. Claude używa odpowiednika test_claude_patterns. Testy sprawdzają także izolację katalogów hostów, odnośniki dokumentacji oraz importy na poziomie modułu.

`agent_demo/check_server.py` startuje dokładny launcher z konfiguracji, będąc w podkatalogu notebooks. Sprawdza negocjację nowoczesną i legacy, listę narzędzi, schemat text, prawidłowe liczenie i odrzucenie brakującego argumentu. Nie jest testem zgody użytkownika.

## Bezpieczne próby natywne

Zamiast naprawdę wykonywać destrukcyjny Git, przekaż taki tekst do handlera albo użyj nieszkodliwego wypisania tekstu, które celowo zostanie rozpoznane. Commit można sprawdzić przez `git commit --dry-run`. Usuwanie testuj wyłącznie na wskazanym artefakcie demonstracyjnym, po wymaganej zgodzie.

W teście subagenta szukaj rzeczywistego rozpoczęcia i zakończenia roli oraz jej wywołania narzędzia. Sama odpowiedź „delegowałem” jest niewystarczająca. W teście skilla upewnij się, że został załadowany, a nie tylko odtworzony z wiedzy modelu.

# 22. Diagnostyka bez zgadywania

| Objaw | Najpierw sprawdź |
|---|---|
| Skill niewidoczny | Katalog, SKILL.md, nagłówek, źródła i sesję |
| Rola niewidoczna | Format TOML/Markdown, wymagane pola, nazwę i runtime |
| Hook nie blokuje | Matcher, aktywność, zaufanie i rzeczywisty tool_input |
| Hook pyta zamiast blokować | Konflikt definicji oraz typ zdarzenia |
| Codex nie pyta o żarty | Per-tool approval_mode, załadowany projekt, użyte narzędzie |
| Claude nie pokazuje pytania | Tryb interaktywny, permission mode i decyzję hooka |
| MCP nie startuje | PATH, uv, katalog, cache, stderr, timeout |
| MCP nie widzi nowej funkcji | Restart serwera i enabled_tools |
| Zapis odmówiony po zgodzie | Istniejący plik, walidację celu, filesystem |
| Działa CLI, nie działa panel | Wersję i środowisko konkretnego hosta |

## Najbardziej zdradliwy przypadek

Masz poprawny plik, poprawny test i otwartą starą rozmowę. Host może nadal używać wcześniejszej konfiguracji albo wymagać ponownego zaufania zmienionemu hookowi. Zamiast mnożyć ustawienia, zacznij świeżą sesję i sprawdź źródła konfiguracji.

Inny częsty błąd to przeniesienie pola między mechanizmami: `tools` subagenta do skilla, `ask` Claude do Codex PreToolUse, albo zmiennej hooka do MCP. Podobne nazwy nie stanowią wspólnego standardu konfiguracji obu aplikacji.

## Raportowanie wyniku

Dobry raport mówi: co zmieniono, gdzie, jaki test przeszedł i czego test nie obejmował. Rozróżnia „monit się pojawił” od „użytkownik zatwierdził i plik powstał”. Przy odmowie podaje przyczynę oraz stan artefaktu. Nie przedstawia braku błędu w logu jako dowodu, że mechanizm był aktywny.

# 23. Plan krótkiego szkolenia

## Część pierwsza: pokaż działanie

Otwórz właściwy branch i README. Wywołaj count_words na trzech słowach. Pokaż funkcję Python oraz jej argument. Następnie użyj explain-code na tej funkcji: uczestnik zobaczy, że skill organizuje wyjaśnienie, ale nie zastępuje narzędzia.

## Część druga: pokaż delegację

Uruchom inspect-lesson dla jednego notebooka. Pokaż definicję roli i jej wynik. Wytłumacz, dlaczego osobny kontekst pomaga utrzymać zadanie w granicach, a dlaczego nie tworzy automatycznie izolacji filesystemu. W Claude porównaj tools reviewera z instrukcjami read-only w Codex.

## Część trzecia: pokaż kontrolę

Zleć joke writerowi utworzenie jokes.txt. Najpierw odrzuć pytanie i sprawdź brak pliku. Potem jawnie uruchom nową próbę, zatwierdź i odczytaj pięć linii. Powiedz, która warstwa pyta: MCP permission w Codex, PreToolUse ask w Claude.

Spróbuj usunąć demonstracyjny plik. Codex ma odmówić i podać pełną ścieżkę. Claude ma przedstawić listę i poczekać. Test zwykłego commita ma zostać odrzucony w obu. Nie pokazuj działania przez globalne wyłączanie zabezpieczeń.

## Pytania kontrolne

• Czy nagłówek skilla definiuje argumenty funkcji MCP? Nie; to różne kontrakty.

• Czy subagent z instrukcją „read-only” zawsze ma technicznie odebrany zapis? Nie; zależy od rzeczywistych uprawnień i narzędzi.

• Czy hook wykonuje zatwierdzaną operację? W naszym wzorcu nie; zwraca decyzję do hosta.

• Czy commit repo przenosi zaufanie na komputer kolegi? Nie.

• Czy `{}` z hooka oznacza „zezwól niezależnie od innych zasad”? Nie; normalne sprawdzanie trwa dalej.

Uczestnik powinien na końcu umieć dodać jeden mały skill, wskazać miejsce roli, opisać argument narzędzia i zidentyfikować warstwę odpowiedzialną za pytanie o zgodę.

# 24. Źródła, wersje i dalsza rozbudowa

Ten przewodnik opisuje konkretne lokalne przykłady i kontrakty sprawdzane 7 listopada 2026. Runtime użyty do próby Codex: 0.164.0-alpha.11.1 z rozszerzenia VS Code. Runtime Claude: 2.1.311. Osobny program z PATH może mieć inną wersję. SDK MCP jest zablokowany w uv.lock projektu agent_demo.

## Oficjalne referencje

- Codex: instrukcje AGENTS.md
- Codex: tworzenie skills
- Codex: subagenci
- Codex: hooki
- Codex: MCP
- Codex: katalog konfiguracji
- Claude: instrukcje i pamięć
- Claude: skills
- Claude: subagenci
- Claude: hooki
- Claude: MCP
- Claude: zakresy i pierwszeństwo
- Claude: pełny indeks ustawień
- MCP: Python SDK

## Jak utrzymywać materiał

Katalogi pól w rozdziałach obejmują mechanizmy potrzebne do tworzenia tych przykładów, w tym główne rozszerzenia skills i subagentów. Pełne aplikacje zawierają znacznie więcej ustawień, np. interfejs, dostawców modeli, sieć, telemetrię i administrację. Indeksy producentów są właściwym miejscem do sprawdzenia kompletnej, zmieniającej się listy tych kluczy.

Przy aktualizacji najpierw sprawdź obsługiwane pola, potem testy handlerów i MCP, na końcu natywne interakcje. Zapisz wynik z wersją hosta i datą. Aktualizuj README docelowego brancha i wspólny dokument pojęciowy; nie twórz kolejnych kopii tej samej instrukcji w wielu folderach.

# 25. Rozszerzony katalog: hooki, narzędzia, pluginy

## Zdarzenia Codex według zastosowania

| Obszar | Nazwy zdarzeń |
|---|---|
| Narzędzia i zgoda | PreToolUse, PostToolUse, PermissionRequest |
| Sesja | SessionStart, SessionEnd |
| Rozmowa | UserPromptSubmit, Stop, Interrupt |
| Kompakcja | PreCompact, PostCompact |
| Delegacja | SubagentStart, SubagentStop |
| Żądanie modelu | PreModelRequest, PostModelRequest |

Nie wszystkie zdarzenia stosują matcher. Dla narzędzi filtrujesz ich nazwę, dla kompakcji jej przyczynę, dla subagenta typ. Zdarzenie zakończenia sesji nie jest tym samym co zakończenie jednej odpowiedzi. W naszym repo aktywowane są tylko PreToolUse i PostToolUse.

## Typy handlerów i dodatkowe pola

| Typ | Pola charakterystyczne |
|---|---|
| command | command, wariant Windows lub args zgodnie z hostem |
| mcp_tool | server, tool, input; dostępny w obu opisanych kontraktach |
| http w Claude | url, headers, allowedEnvVars, onFailure |
| prompt / agent w Claude | prompt, opcjonalny model; zgodność zależy od zdarzenia |

Codex rozpoznaje command i mcp_tool; prompt i agent nie są w tym kontrakcie wykonywane. W hooku mcp_tool argument input może korzystać z pól eventu, np. `${tool_input.file_path}`. To wywołanie uruchamiane przez hook, a nie przez model.

**Nie podłączaj write_jokes jako wykonawczego hooka MCP.** W Codex taki hook nie przechodzi zwykłego pytania o zgodę narzędzia. Nasz wzorzec wymaga, aby model/subagent proponował zwykłe wywołanie MCP, objęte approval_mode prompt. Podobna nazwa integracji nie oznacza identycznej ścieżki uprawnień.

## Parametry dekoratora naszego SDK

Sprawdzona lokalnie sygnatura `MCPServer.tool` dopuszcza name, title, description, annotations, icons, meta i structured_output. Domyślnie nazwę i opis można wyprowadzić z funkcji. annotations opisują cechy operacji; nie zastępują walidacji ani nie dowodzą, że obcy serwer mówi prawdę. structured_output dotyczy formy wyniku, nie prawa zapisu.

# 26. Przepisy na zakres repo, user i machine

## Repo: przenośny zespół

W Codex dopisz małą definicję do projektowego config.toml i odpowiedni plik skilla lub roli. W Claude dopisz do settings.json, a MCP do .mcp.json. Nie kopiuj całej osobistej konfiguracji do repo: zawiera preferencje, których inni nie powinni dziedziczyć.

Po zmianie sprawdź diff, składnię i świeży host. Osoba wdrażająca ręcznie tworzy commit; w naszej polityce agent go nie wykonuje. Następnie drugi użytkownik pobiera branch, odtwarza środowisko agent_demo i sam zatwierdza zaufanie.

## User: osobista preferencja

Codex: edytuj `~/.codex/config.toml` albo właściwy CODEX_HOME. Profile są osobnymi plikami `NAZWA.config.toml` obok niego, wybieranymi przez `--profile NAZWA`. W tym kontrakcie projekt nie może przejmować m.in. model_provider, model_providers, notify, profile ani routingu otel; te wybory należą do odpowiedniego zakresu użytkownika/hosta.

Claude: osobista konfiguracja wielu projektów jest w `~/.claude/settings.json`. Wyjątek tylko dla jednego checkoutu zapisuj w `.claude/settings.local.json`. Przykład osobistego ustawienia należy najpierw sprawdzić w katalogu kluczy; nie każdy klucz aplikacji dopuszcza zakres project.

## Machine i organizacja

Na Windows administracyjny plik Claude to `C:/Program Files/ClaudeCode/managed-settings.json`; na Linux/WSL używa się `/etc/claude-code/managed-settings.json`, a na macOS `/Library/Application Support/ClaudeCode/managed-settings.json`. Organizacja może dostarczać politykę także innym wspieranym kanałem. Stary katalog ProgramData nie jest równoważnym zamiennikiem.

Codex rozdziela zwykłą konfigurację i administracyjne requirements.toml. Managed hooks mogą wskazywać osobny katalog instalowany przez administratora. `allow_managed_hooks_only` oraz przypięcie features.hooks służą do kontrolowania źródeł hooków, ale wymagają właściwego wdrożenia, nie dopisku w dowolnym repo.

Nie wdrażaliśmy tutaj polityk całego komputera. Ten rozdział pokazuje, gdzie należy je projektować. Polityka Claude na urządzeniu ma odrębne zasady dostarczania i weryfikacji od ustawień zespołu zapisanych w Git.

# 27. Stan sprawdzenia tej konfiguracji

Poniższy zapis dotyczy przygotowanych checkoutów z 7 listopada 2026. Wszystkie opisane zgody zostały udzielone, a skutki operacji sprawdzone. Zmiany pozostają lokalne i niezacommitowane; agent nie tworzy commitów.

| Kontrola | Zaobserwowany wynik |
|---|---|
| Codex: testy jednostkowe i układ importów | 19 testów zakończonych powodzeniem |
| Claude: testy jednostkowe i układ importów | 15 testów zakończonych powodzeniem |
| MCP w obu checkoutach | Discovery, oba schematy, liczenie, błędne wejścia: poprawnie; auto i legacy |
| Skills w obu hostach | explain-code, review-lesson i inspect-lesson użyte na plikach repo |
| Reviewerzy w obu hostach | Przegląd dokumentacji i lekcji zakończony; bez wykonania notebooka |
| Zwykły commit | Hook odmówił w obu hostach |
| Rozpoznany destrukcyjny Git | Hook odmówił w obu hostach |
| Codex: usunięcie jokes.txt | Odmowa z pełną ścieżką; istniejący plik zachowany |
| Codex: joke_writer | Natywna zgoda MCP; po akceptacji subagent zapisał i odczytał 5 żartów |
| Claude: usunięcie jokes.txt | Pytanie hooka z pełną ścieżką; po zgodzie plik usunięty |
| Claude: joke-writer | Osobna zgoda z hooka; subagent zapisał i odczytał 5 nowych żartów |
| Ochrona istniejącego pliku | Oba walidatory odmówiły nadpisania; sumy SHA-256 bez zmian |

## Co potwierdzono w sesjach

W obu hostach zapis wykonało narzędzie wywołane przez subagenta po zgodzie użytkownika. Rodzic przekazał wybór do istniejącego monitu, ale nie zastąpił subagenta przy zapisie. Oba końcowe pliki mają po pięć ponumerowanych linii. Plik Codex pozostał po zablokowanej próbie usunięcia.

Testy natywne używały CLI dostarczanego z rozszerzeniami. Nie są dowodem kliknięć w graficznym panelu VS Code. Składnia, odkrywanie konfiguracji i mechanizmy runtime są sprawdzone w opisanym zakresie; zachowanie konkretnego panelu należy odnotować osobno, jeśli to on ma być przedmiotem szkolenia.

## Znane ograniczenia poza przykładem

Codex zgłaszał ignorowane osobiste ustawienie `computer_use.windows.always_allowed_app_ids`. Nie pochodzi ono z tych branchów i nie zmienialiśmy go w ramach tej pracy. W Claude pozostaje stary, niezarejestrowany ask_before_change.py; aktywną politykę definiuje change_policy.py. Nie należy uruchamiać starego handlera jako reprezentacji obecnego przykładu.

# Aneks: pliki codex

Pełne definicje z przygotowanego checkoutu. Długie wiersze są zawijane wyłącznie na potrzeby wydruku; gotowy plik kopiuj z repo. To migawka konfiguracji, nie zapis osobistych sekretów ani uprawnień użytkownika.

## .codex/config.toml

```
# Credentials, model preferences, and trust remain on the host.
[agents]
enabled = true
max_concurrent_threads_per_session = 2

[mcp_servers.lesson_tools]
command = "powershell.exe"
args = ["-NoProfile", "-NonInteractive", "-Command", "$ErrorActionPreference =
'Stop'\n$repo = git rev-parse --show-toplevel\nif ($LASTEXITCODE -ne 0) { throw 'Open a
checkout of the agent examples first.' }\n& uv run --frozen --project \"$repo/agent_demo\"
python \"$repo/agent_demo/server.py\"\nexit $LASTEXITCODE"]
enabled_tools = ["count_words", "write_jokes"]
startup_timeout_sec = 30
tool_timeout_sec = 10

# The host prompts before every joke write, including subagent calls.
[mcp_servers.lesson_tools.tools.write_jokes]
approval_mode = "prompt"
```

## .codex/hooks.json

```
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "^(Bash|exec_command)$",
        "hooks": [
          {
            "type": "command",
            "command": "python -I \"$(git rev-parse --show-
toplevel)/.codex/hooks/git_safety.py\"",
            "commandWindows": "python -I \"$(git rev-parse --show-
toplevel)/.codex/hooks/git_safety.py\"",
            "timeout": 10,
            "statusMessage": "Checking destructive Git commands"
          }
        ]
      },
      {
        "matcher": ".*",
        "hooks": [
          {
            "type": "command",
            "command": "python -I \"$(git rev-parse --show-
toplevel)/.codex/hooks/change_policy.py\" --deletion-action deny",
            "commandWindows": "python -I \"$(git rev-parse --show-
toplevel)/.codex/hooks/change_policy.py\" --deletion-action deny",
            "timeout": 10,
            "statusMessage": "Checking commits and deletion targets"
          }
        ]

      },
      {
        "matcher": "^mcp__lesson_tools__write_jokes$",
```

```
        "hooks": [
          {
            "type": "command",
            "command": "python -I \"$(git rev-parse --show-
toplevel)/.codex/hooks/approve_jokes.py\" --valid-action continue",
            "commandWindows": "python -I \"$(git rev-parse --show-
toplevel)/.codex/hooks/approve_jokes.py\" --valid-action continue",
            "timeout": 10,
            "statusMessage": "Validating the proposed joke write"
          }
        ]
      }
    ],
    "PostToolUse": [
      {
        "matcher": "^(apply_patch|Edit|Write)$",
        "hooks": [
          {
            "type": "command",
            "command": "python -I \"$(git rev-parse --show-
toplevel)/.codex/hooks/notebook_context.py\"",
            "commandWindows": "python -I \"$(git rev-parse --show-
toplevel)/.codex/hooks/notebook_context.py\"",
            "timeout": 10,
            "statusMessage": "Adding notebook validation context"
          }
        ]

      }
    ]
  }
}
```

## .codex/agents/docs_reviewer.toml

```
name = "docs_reviewer"
description = "Check named repository documents against actual code and configuration
without editing."
developer_instructions = """
Read only the supplied documents and evidence needed to verify their claims.
Check paths, working directories, option names, and validation claims.
Return confirmed mismatches with document and implementation locations,
plus small proposed corrections. Distinguish uncertainty from an actual error.
Do not execute documented examples, call services, or edit files for this review.
"""
```

## .codex/agents/joke_writer.toml

```
name = "joke_writer"
description = "Propose five fresh jokes and create root jokes.txt through native MCP
approval."
developer_instructions = """
Read root AGENTS.md. You own the requested MCP call and readback.
If root jokes.txt exists, preserve it and stop.
Generate five fresh, friendly programming jokes in the requested language,
defaulting to English. Call lesson_tools.write_jokes with the absolute root
jokes.txt file_path and a jokes array of five unnumbered single-line strings.
The host must request native approval before execution. Do not ask the parent
to write the file or record approval. Never use another write tool or script.
After success read the file and verify five numbered lines. If refused, stop.
Do not delete files or commit. You inherit parent tools and permissions.
"""
```

## .codex/agents/lesson_reviewer.toml

```
name = "lesson_reviewer"
description = "Review named teaching notebooks for conceptual jumps and missing
prerequisites."
developer_instructions = """
Read AGENTS.md and the specified notebooks in order. Use the supplied audience
and prerequisites, or assume basic Python. Find the first confusing step and
explain the student's likely question. Give a path, 1-based cell number, and
smallest useful correction for each material finding. Do not invent problems.
Do not edit files, run notebooks, or call model services. Report that this is
a reading review, not execution proof.
"""
```

## .agents/skills/explain-code/SKILL.md

```
---
name: explain-code
description: Explain specified repository code or notebooks through an example and
execution flow.
---

Read the named files and applicable project instructions. Start with the problem,
trace one small input through the real code, and show the result. Cite paths,
symbols, or notebook cell numbers. Explain new terms where they first appear.

Use the audience, focus, and detail level supplied in the request. If omitted,
assume a reader who knows basic Python but not this framework. Follow dependencies
only as needed. Distinguish reading, inference, and execution evidence.
An explanation does not require running notebooks, model services, or editing files.
```

## .agents/skills/inspect-lesson/SKILL.md

```
---
name: inspect-lesson
description: Delegate a notebook learning-flow review to the lesson_reviewer specialist.
---

Take a notebook path and intended audience from the request. Ask for either
missing value. Delegate to lesson_reviewer with that path, audience, and any
relevant user constraints; the specialist should read the notebook and root
AGENTS.md. Request confirmed confusing steps with cell references and small
corrections. Report the review without editing or executing the notebook.
```

## .agents/skills/review-lesson/SKILL.md

```
---
name: review-lesson
description: Review named teaching notebooks for missing prerequisites and confusing
learning steps.
---

Read the requested notebooks in the given order and the relevant project rules.
Use the audience and prerequisites from the request; otherwise assume basic Python.
Identify the first unexplained term, hidden state dependency, or conceptual jump.
For each material issue give a file, 1-based cell number, likely student question,
and the smallest improvement. Report strengths when useful; do not fill a quota.

Review only by reading unless execution was requested. Do not invoke services or
run kernels to check clarity. Return the review without edits unless editing is
part of the request. Delegation is optional, not required by this shared workflow.
```

# Aneks: pliki claude

Pełne definicje z przygotowanego checkoutu. Długie wiersze są zawijane wyłącznie na potrzeby wydruku; gotowy plik kopiuj z repo. To migawka konfiguracji, nie zapis osobistych sekretów ani uprawnień użytkownika.

## .claude/settings.json

```
{
  "permissions": {
    "defaultMode": "default"
  },
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "^(Bash|PowerShell)$",
        "hooks": [
          {
            "type": "command",
            "command": "python",
            "args": [
              "-I",
              "${CLAUDE_PROJECT_DIR}/.claude/hooks/git_safety.py"
            ],
            "timeout": 10,
            "onFailure": "block"
          }
        ]
      },
      {
        "matcher": ".*",
        "hooks": [
          {
            "type": "command",
            "command": "python",
            "args": [
              "-I",
              "${CLAUDE_PROJECT_DIR}/.claude/hooks/change_policy.py",

              "--deletion-action",
              "ask"
            ],
            "timeout": 10,
            "onFailure": "block"
          }
        ]
      },
      {
        "matcher": "^mcp__lesson_tools__write_jokes$",
        "hooks": [
          {
            "type": "command",
            "command": "python",
            "args": [
              "-I",
              "${CLAUDE_PROJECT_DIR}/.claude/hooks/approve_jokes.py",
              "--valid-action",
              "ask"
            ],
            "timeout": 10,
            "onFailure": "block"
          }
        ]
      }
```

```
    ],
    "PostToolUse": [
      {
        "matcher": "^(Edit|Write|NotebookEdit)$",
        "hooks": [

          {
            "type": "command",
            "command": "python",
            "args": [
              "-I",
              "${CLAUDE_PROJECT_DIR}/.claude/hooks/notebook_context.py"
            ],
            "timeout": 10,
            "onFailure": "continue"
          }
        ]
      }
    ]
  }
}
```

## .mcp.json

```
{
  "mcpServers": {
    "lesson_tools": {
      "type": "stdio",
      "command": "powershell.exe",
      "args": [
        "-NoProfile",
        "-NonInteractive",
        "-Command",
        "$ErrorActionPreference = 'Stop'\n$repo = git rev-parse --show-toplevel\nif
($LASTEXITCODE -ne 0) { throw 'Open a checkout of the agent examples first.' }\n& uv run
--frozen --project \"$repo/agent_demo\" python \"$repo/agent_demo/server.py\"\nexit
$LASTEXITCODE"
      ]
    }
  }
}
```

## .claude/agents/docs-reviewer.md

```
---
name: docs-reviewer
description: Check named repository documentation against actual files and configuration.
tools: Read, Grep, Glob
model: inherit
---

Read root AGENTS.md and the named documentation. Follow its file references and
compare described behavior with actual configuration or implementation. Return
confirmed discrepancies with paths and line references, why each matters, and
a minimal correction. Do not edit files or execute commands. Say when evidence
is insufficient. Do not invent findings to meet a quota.
```

## .claude/agents/joke-writer.md

```
---
name: joke-writer
description: Propose five fresh jokes and create root jokes.txt through native MCP
approval.
tools: Read, mcp__lesson_tools__write_jokes
model: inherit
permissionMode: default
---

Read root AGENTS.md. You own the requested MCP call and readback.
If root jokes.txt exists, preserve it and stop.
Generate five fresh, friendly programming jokes in the requested language,
defaulting to English. Call lesson_tools.write_jokes with the absolute root
jokes.txt file_path and a jokes array of five unnumbered single-line strings.
The project hook validates the call and asks through Claude's native UI.
Do not ask the parent to write the file or record approval. Do not switch tools.
After success read the file and verify five numbered lines. If refused, stop.
Do not delete files or commit.
```

## .claude/agents/lesson-reviewer.md

```
---
name: lesson-reviewer
description: Review named teaching notebooks for missing prerequisites and conceptual
jumps.
tools: Read, Grep, Glob
model: inherit
---

Read AGENTS.md and only the requested notebooks plus necessary prerequisites.
Use the supplied audience, or assume basic Python. Return material learning
obstacles with a path, 1-based cell number, likely student question, and the
smallest improvement. Do not manufacture findings. This is a reading review;
do not claim runtime verification.
```

## .claude/skills/explain-code/SKILL.md

```
---
name: explain-code
description: Explain specified repository code or notebooks through an example and
execution flow.
---

Read the named files and applicable project instructions. Start with the problem,
trace one small input through the real code, and show the result. Cite paths,
symbols, or notebook cell numbers. Explain new terms where they first appear.

Use the audience, focus, and detail level supplied in the request. If omitted,
assume a reader who knows basic Python but not this framework. Follow dependencies
only as needed. Distinguish reading, inference, and execution evidence.
An explanation does not require running notebooks, model services, or editing files.
```

## .claude/skills/inspect-lesson/SKILL.md

```
---
name: inspect-lesson
description: Explicitly delegate a notebook learning-flow review to lesson-reviewer.
argument-hint: "<notebook-path> <audience>"
disable-model-invocation: true
context: fork
background: false
agent: lesson-reviewer
---

Review notebook $0 for audience $1.
If either argument is missing or a placeholder remains unresolved, request the
missing value before reviewing. Read the notebook without executing or editing.
Return material learning obstacles with cell references and small corrections.
```

## .claude/skills/review-lesson/SKILL.md

```
---
name: review-lesson
description: Review named teaching notebooks for missing prerequisites and confusing
learning steps.
---

Read the requested notebooks in the given order and the relevant project rules.
Use the audience and prerequisites from the request; otherwise assume basic Python.
Identify the first unexplained term, hidden state dependency, or conceptual jump.
For each material issue give a file, 1-based cell number, likely student question,
and the smallest improvement. Report strengths when useful; do not fill a quota.

Review only by reading unless execution was requested. Do not invoke services or
run kernels to check clarity. Return the review without edits unless editing is
part of the request. Delegation is optional, not required by this shared workflow.
```